BCSE329L — BLOCKCHAIN AND DISTRIBUTED LEDGER TECHNOLOGY

Authentimed

AI + Blockchain Based Counterfeit Drug Detection System

Review 2 | Implementation Progress Report

Team

Arnav Sinha (23BKT0028)
Kapilan Kathirvel (23BKT0035)
Ashwin Raj (23BKT0103)

Faculty Guide

Ms. Sivaranjani A

Date

10 & 17 Sept 2026

01 RECAP

Recap of Review 1 — The Problem

1 in 10

medical products in LMICs is substandard or falsified (WHO)

$30.5 bn

spent globally every year on falsified / substandard medicine

>1 million

deaths per year attributed to these products (WHO)

Research gap identified in Review 1

Static QR / barcode mandates (India Schedule H2, US DSCSA, EU FMD) make serialization compulsory — but a printed code carries no secret. A photograph of one genuine pack (as in the Feb 2025 Howrah seizure) can be reprinted on thousands of fakes, and a stateless scan only checks that a code exists, never whether that pack has already been sold.

Our framework's contribution: a stateful custody ledger where every pack carries a lifecycle state, not merely a registered identity — so duplication itself becomes detectable.

01 RECAP

What's Changed Since Review 1

1 Full-Stack Implementation

React frontend + Flask backend + Sepolia Testnet smart contract — a live deployed end-to-end system replacing the Review 1 design-only proposal.

2 AI Packaging Verification

OpenCV-based structural comparison checks the physical authenticity of packaging — extending scope from digital-only to digital + physical validation.

3 Dual-Factor Authentication

QR-code serialization plus an embedded 10-char hidden PAN code (revealed via colour-channel processing) linked in Supabase PostgreSQL for stronger physical security.

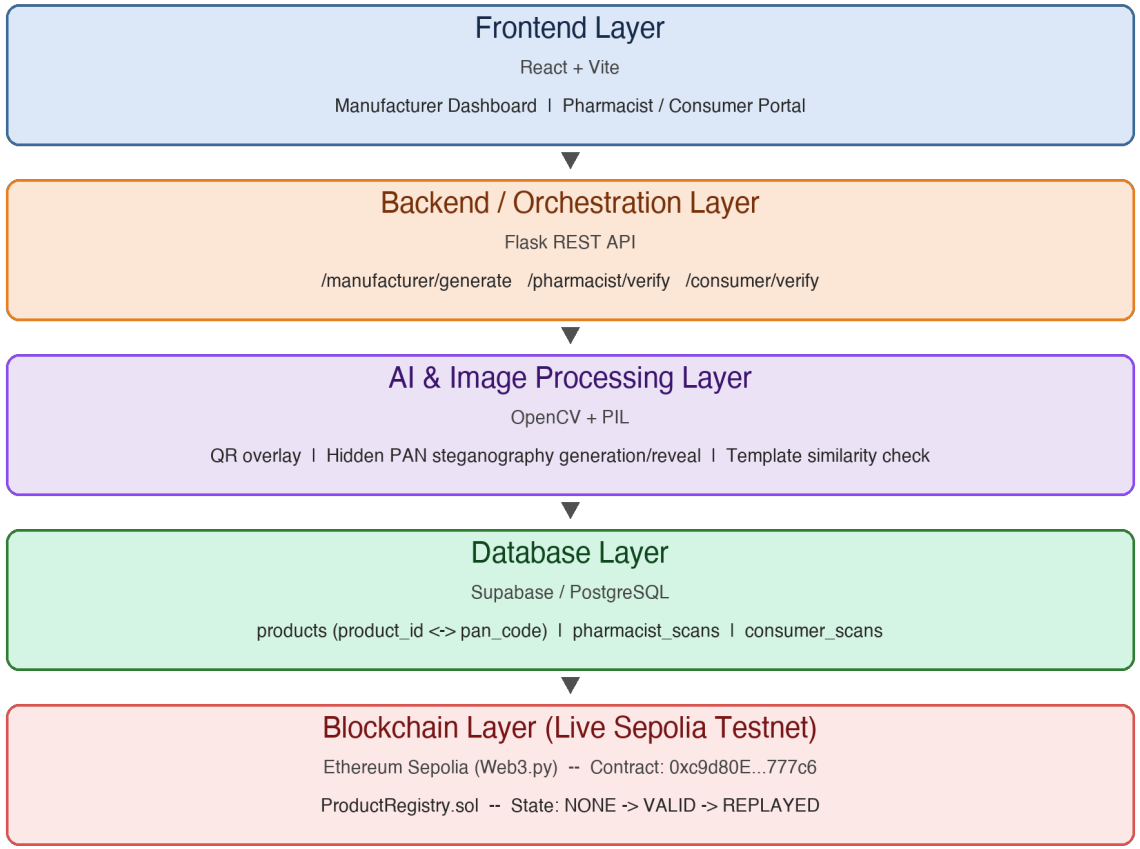
4 Stateful Replay Protection

The core Review-1 anti-cloning mechanism is implemented end-to-end: one scan per product, enforced on-chain.

System Architecture

Five layers, deployed end-to-end

React/Vite frontend, a Flask orchestration backend, an OpenCV + PIL AI layer, Supabase/PostgreSQL for off-chain records, and an Ethereum Sepolia smart contract (ProductRegistry.sol) as single source of truth.

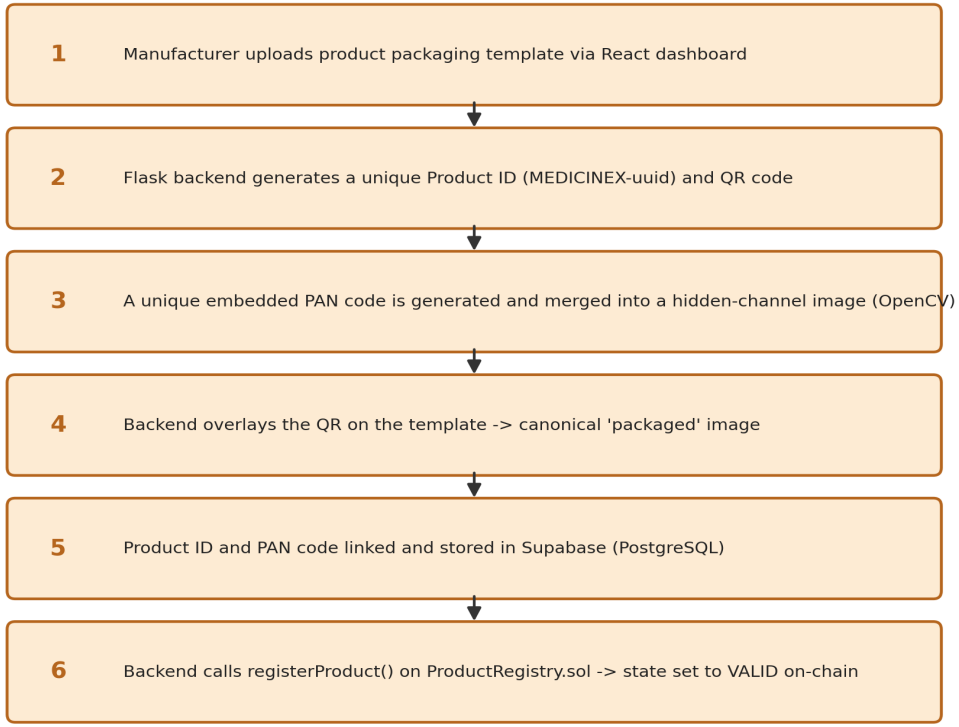


Key point

Only the fingerprint and lifecycle state live on-chain. Image data and similarity checks stay off-chain — the Review 1 design principle, now implemented.

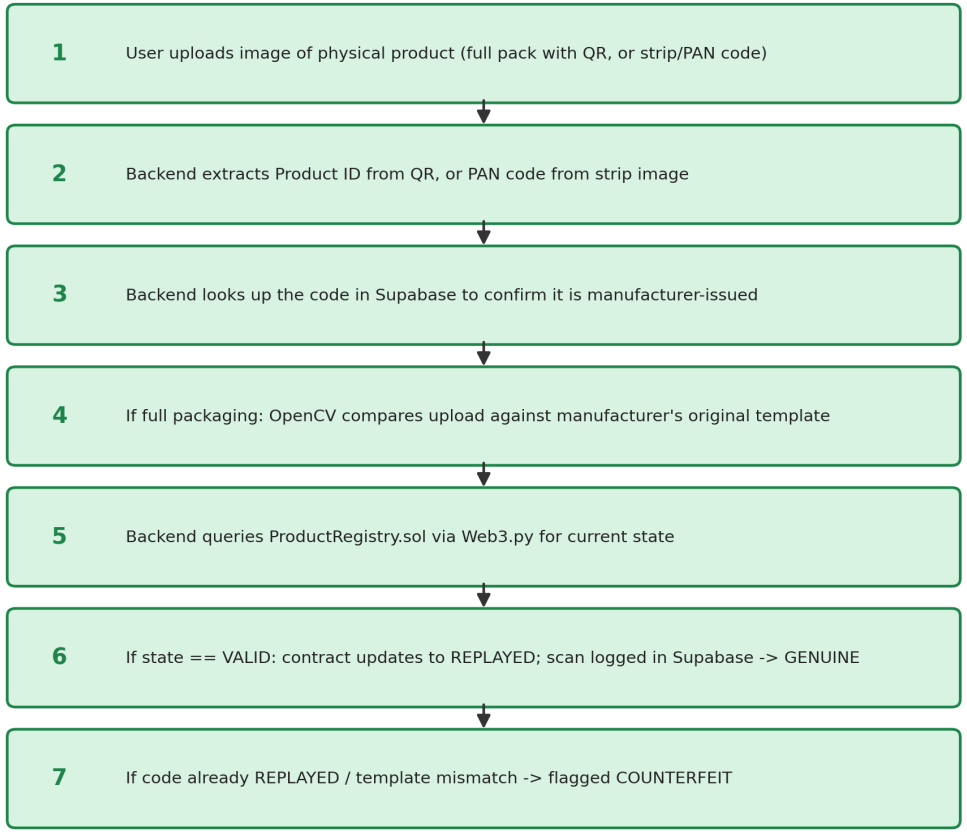
Workflow — Manufacturer Registration

The manufacturer uploads a packaging template. The backend mints a unique Product ID (MEDICINEX-uuid), generates a QR code and a hidden PAN code, produces the canonical packaged image, stores the mapping in Supabase, and registers the product on-chain in the VALID state.



Output: canonical packaged image + on-chain VALID state

Workflow — Verification & Replay Detection



A pharmacist or consumer uploads a photo. The backend extracts the identifier, checks Supabase, runs the OpenCV template comparison for full-pack images, then queries the smart contract.

VERIFICATION LOGIC (backend/app.py):

```
existing = get_pharmacist_scan(product_id)
if existing:
  return jsonify({'verdict': 'COUNTERFEIT'}), 200
tx = verify_product(product_id) # Sepolia call
if tx['success']:
  record_pharmacist_scan(product_id)
  return jsonify({'verdict': 'GENUINE'}), 200
```

If already REPLAYED, or template mismatch -> COUNTERFEIT

A photographed and reprinted QR passes a stateless, database-only check — but is caught here the moment it is scanned a second time, because state lives on-chain.

Dataset Details

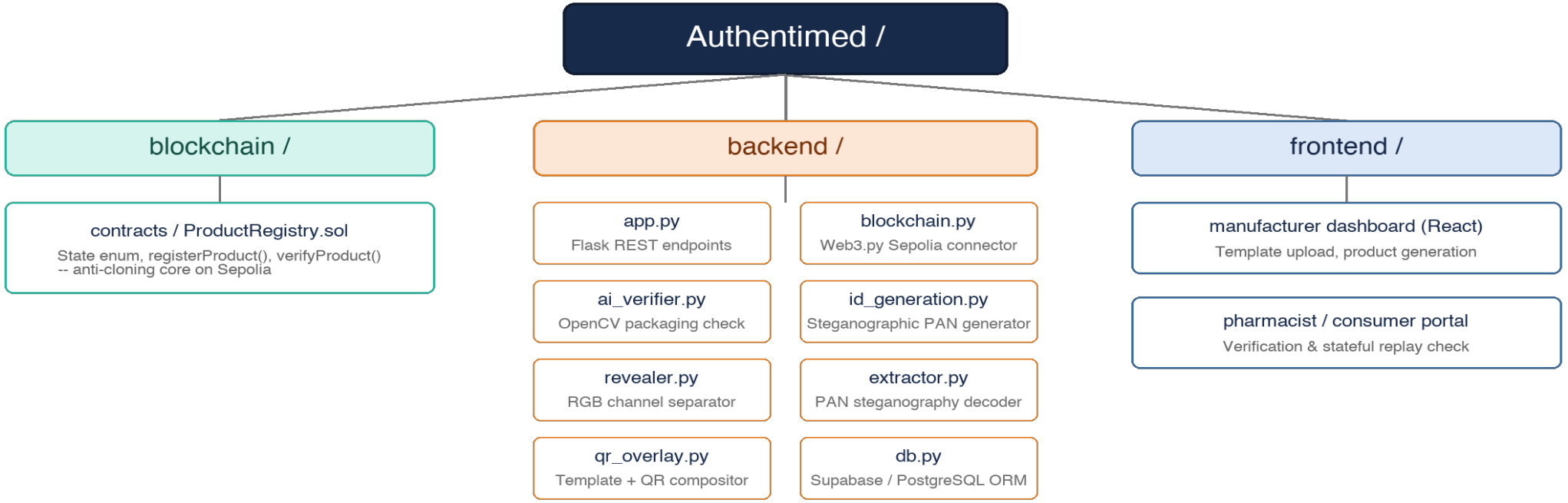
No pre-collected external dataset is used. The verification task's 'ground truth' is each manufacturer's own registered template, so data is generated dynamically by the system itself.

Source	Manufacturer-uploaded packaging templates + dynamically generated QR / PAN codes

Preprocessing	**OpenCV AI (ai-auth/verify.py):**
	`scan_blur, scan_col, scan_cent = extract_features(scan_img)`
	`tpl_blur, tpl_col, tpl_cent = extract_features(template_img)`
	`if scan_blur < tpl_blur * 0.7: return False # Blur check`
	`if norm(scan_col - tpl_col) > 80.0: return False # Color check`
	`if norm(scan_cent - tpl_cent) > 12.0: return False # Edge check`

Storage	Packaged, hidden, and R/G/B reveal images are written to backend/generated/; only file references (not binary data) are passed to the frontend
Splits	No train/validation/test split — verification is a one-shot comparison against a single registered template, not a trained classifier

Implementation — Code Structure



Core Innovation — Anti-Cloning State Machine



Every registered product exists in exactly one on-chain state. A product may move VALID -> REPLAYED exactly once, inside ProductRegistry.sol — this single guarded transition is what turns a copied / reprinted QR code from an undetectable clone into an immediate COUNTERFEIT flag on its second use.

Stateless QR check

confirms a code exists

Stateful ledger check

confirms it hasn't been claimed

verifyProduct() — the guarded state transition

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

enum State { NONE, VALID, REPLAYED }

function registerProduct(string memory id) public onlyOwner {
    require(products[id].state == State.NONE, "Already registered");
    products[id] = Product({ manufacturer: msg.sender, state: State.VALID });
}

// Anti-cloning core: single guarded transition VALID -> REPLAYED
function verifyProduct(string memory id) public returns (bool) {
    if (products[id].state == State.VALID) {
        products[id].state = State.REPLAYED; // mark claimed
        return true; // first scan: GENUINE
    }
    return false; // repeat scan: COUNTERFEIT
}
```

One-way state transition enforced on-chain; state lives in ProductRegistry.sol on Sepolia, preventing re-validation.

Dual-Factor Security — Hidden PAN Code

1 **Generate PAN**

A random 10-character packaging authentication number (4 letters + 5 digits + 1 letter, e.g. ABCD12345E) is generated per product at registration.

2 **Embed in colour channel**

The 10-char PAN is validated (_validate_code: 4 letters + 5 digits + 1 letter) & bit-encoded:

```
def _validate_code(code): # 10-char validation
  if len(code)!=10 or not (code[:4].isalpha() and code[4:9].isdigit()):
    raise ValueError('Format: 4 letters + 5 digits + 1 letter')
  return code.upper()
```

3 **Link to Product ID**

QR-derived Product ID and PAN code are stored as a linked pair in Supabase (products table), so either factor verifies the same identity.

4 **Reveal on scan**

A strip scan extracts channels (revealer.py) and decodes segment bit patterns (extractor.py) back into the PAN string.

Why it matters: photographing visible packaging alone does not disclose the PAN factor — raising the bar beyond a simple QR copy-and-reprint attack.

Testing & Validation Results

Unit · Integration · Security / Adversarial testing performed on the running pipeline

#	Scenario	Expected	Actual
1	Generate product	Success — on-chain registration	Success
2	First pharmacist scan (genuine QR)	GENUINE	GENUINE
3	Duplicate scan (replay attack)	COUNTERFEIT — replay detected	COUNTERFEIT — duplicate scan
4	Valid QR on incorrect packaging	COUNTERFEIT — packaging check failed	COUNTERFEIT — AI validation failed

Final Evaluation Plan

Metrics defined in the Review 1 proposal, and how each will be measured for Review 3

Evaluation Parameter	Planned Measurement Method
Verification correctness / clone detection rate	Scripted batch of genuine + duplicated QR submissions against the deployed contract; count correct GENUINE / COUNTERFEIT verdicts
False positive rate on legitimate rescans	Repeat legitimate manufacturer / regulator audit scans through a dedicated read-only path; confirm no false COUNTERFEIT flags
Gas cost per registration & verification	Deploy to Sepolia testnet; log gas used per registerProduct() / verifyProduct() call across a batch
End-to-end verification latency	Timed requests from image upload to verdict, across the AI check + contract call round trip
Scalability at realistic pack volumes	Load-test the Flask API with concurrent verification requests; measure throughput degradation
Usability (pharmacist / patient)	Structured walkthrough with test users on the React dashboards; task-completion time and error rate

Plan for Outcome

1

Public testnet deployment (COMPLETED)

ProductRegistry.sol deployed on Sepolia (0xc9d80E...777c6) via Web3.py in backend/blockchain.py.

2

Quantified results package

Gas cost, latency, clone-detection and false-positive tables ready for Review 3's comparison section.

3

Role-based access control

JWT-authenticated RBAC across regulator / manufacturer / distributor / pharmacist / patient roles.

4

Baseline comparison

Benchmark against static-QR / stateless-portal approaches (e.g. MediLedger-style prior art) named in Review 1.

5

IoT cold-chain extension

Bring in the proposed Layer-2 sensing pipeline so substandard (not just falsified) products can be flagged.

6

Multi-hop custody binding

Extend beyond direct manufacturer-to-pharmacist transfer to a full signed custody chain.

Thank You

Questions & Discussion

GitHub Repository

https://github.com/BruhGunned/Authentimed

Arnav Sinha (23BKT0028) · Kapilan Kathirvel (23BKT0035) · Ashwin Raj (23BKT0103)